

DATABASE MANAGEMENT SYSTEMS

B.Tech IV Semester



Prepared
By
Dr V.SITHARAMULU

GITAM (Deemed to be University)

GITAM, Rudraram, Patancheru mandal, Hyderabad - 502329 Telangana, India

MODULE-I : INTRODUCTION TO DBMS AND DATABASE DESIGN

Introduction to DBMS: File system vs DBMS, advantages of DBMS, storage data, queries, DBMS structure, Types of Databases– Hierarchical, Network, Relational, Key-Value, Object-oriented, XML DB.

Overview of File Structures in database.

E-R model: Entities, attributes and entity sets, relationship and relationship set, mapping cardinalities, keys, features of ER model, conceptual database design with ER model.

MODULE-II : RELATIONAL MODEL AND BASIC SQL

Relational model: Integrity constraints over relations and enforcement, querying relational data, logical database design, views, destroying/altering tables and views.

Basic SQL: Introduction to SQL, Basic SQL Queries: DML, DDL, DCL, TCL

MODULE-III ADVANCES SQL AND PL/SQL

Structured Query Language (SQL): Select Commands, Union, Intersection, Except, Nested Queries, Aggregate Operators, Null values, Relational set operators, SQL join operators

Relational Algebra(RA): Selection, Projection, Set operations, Joins

Relational Calculus (TRC, DRC): Tuple Relational Calculus, Domain Relational Calculus

PL/SQL, Assertions, Triggers

MODULE-IV : SCHEMA REFINEMENT AND NORMAL FORMS

Introduction to Schema Refinement, Functional Dependencies, Reasoning about Functional Dependencies. Normal Forms, Properties of Decomposition, Normalization, different types of dependencies.

MODULE-V INTRODUCTION TO TRANSACTION MANAGEMENT, CONCURRENCY CONTROL AND CRASH RECOVERY

Introduction to Transaction Management: ACID properties, Transactions and Schedules, Concurrent Execution of Transactions, Lock-Based Concurrency Control.

Concurrency Control: 2PL, Serializability and Recoverability, Introduction to Lock Management, Lock Conversions, Dealing with Deadlocks, Concurrency control without locking.

Crash Recovery: Aries, Recovering from a System Crash

TEXT BOOKS:

1. Raghu Ramakrishnan and Johannes Gehrke, Database Management Systems, 3rd Edition, McGraw-Hill, 2014, 978-8131769591.
2. H.F.Korth and A.silberschatz, Database System Concepts, 7th Edition, McGraw-Hill, 2021, 978-9390727506.

REFERENCE BOOKS:

1. D. Ullman,, Principles of Database and Knowledge – Base Systems, Vol 1, 1st Edition, Computer Science Press, 1990
2. Ramez Elmasri, Shamkant B. Navathe, Fundamentals of Database Systems, Pearson Education, 2016

Q1. Display all columns of EMP table

```
SELECT * FROM EMP;
```

```
SQL> SET PAGESIZE 50
SQL> select * from emp;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1001	RAJESH	MANAGER		10-JAN-23	65000		10
1002	SURESH	CLERK	1001	15-FEB-23	25000		20
1003	MAHESH	ANALYST	1001	18-MAR-23	48000		10
1004	KIRAN	SALESMAN	1003	05-APR-23	30000	2000	30
1005	RAMESH	CLERK	1003	12-MAY-23	22000		20
1006	SANTHOSH	MANAGER		20-JUN-23	70000		40
1007	AMITH	SALESMAN	1006	10-JUL-23	32000	1500	30
1008	DEEPAK	ANALYST	1006	15-AUG-23	52000		40
1009	ANU	CLERK	1006	25-SEP-23	24000		20
1010	PRIYA	MANAGER		01-OCT-23	68000		10
1011	VISHAL	SALESMAN	1010	05-NOV-23	31000	3000	30
1012	NAVEEN	ANALYST	1010	12-DEC-23	50000		40
1013	HARISH	CLERK	1012	18-JAN-24	23000		20
1014	MEENA	SALESMAN	1012	20-FEB-24	29000	2500	30
1015	DIVYA	MANAGER		10-MAR-24	72000		50
1016	ARUN	CLERK	1015	22-APR-24	21000		20
1017	SANDYA	ANALYST	1015	11-MAY-24	53000		40
1018	KAVYA	SALESMAN	1015	15-JUN-24	30000	1800	30
1019	VARUN	CLERK	1017	18-JUL-24	22000		20
1020	MANOJ	ANALYST	1017	12-AUG-24	54000		40

```
20 rows selected.
```

Set operations

Set operations are a set of SQL commands that allow you to combine the results of two or more SELECT statements. The three most common set operations are UNION, INTERSECT, and MINUS/EXCEPT.

- **UNION:** It combines the results of two or more SELECT statements and returns only the unique rows.
- **UNION ALL:** Combines results. Includes duplicate rows.
- **INTERSECT:** It returns only the rows that are common to both SELECT statements.
- **MINUS (Oracle) / EXCEPT (SQL Server, MySQL 8+):** It returns only the rows that are unique to the first SELECT statement/Returns rows from first query not present in second query.

Set operations

To display all department numbers from EMP and DEPT (without duplicates)

```
SELECT deptno FROM emp  
UNION  
SELECT deptno FROM dept;
```

To display all department numbers from EMP and DEPT including duplicates

```
SELECT deptno FROM emp  
UNION ALL  
SELECT deptno FROM dept;
```

To display department numbers that are in DEPT table but not in EMP table.

```
SELECT deptno FROM dept  
MINUS  
SELECT deptno FROM emp;
```

To display department numbers that are present in both EMP and DEPT tables.

```
SELECT deptno FROM emp  
INTERSECT  
SELECT deptno FROM dept;
```


Set operations

Display employee names of department 10 and department 20 using set operators.

Display employee names who are MANAGER or ANALYST using set operator.

Operator	Removes Duplicates	Returns
UNION	Yes	Combined rows
UNION ALL	No	Combined rows
INTERSECT	Yes	Common rows
MINUS	Yes	Difference rows

Aggregate Operators

Aggregate Functions (Aggregate Operators) in SQL Aggregate functions perform calculations on a group of rows and return one single value.

Types of Aggregate Functions

Function

COUNT()

SUM()

AVG()

MAX()

MIN()

Description

Counts number of rows

Calculates total value

Calculates average value

Returns maximum value

Returns minimum value

Aggregate Operators

Count total employees

```
SELECT COUNT(*) FROM emp;
```

Count employees in department 10

```
SELECT COUNT(*) FROM emp WHERE deptno = 10;
```

Total salary of all employees

```
SELECT SUM(sal) FROM emp;
```

Total salary of department 20

```
SELECT SUM(sal) FROM emp WHERE deptno = 20;
```

Lowest salary

```
SELECT MIN(sal) FROM emp;
```

Average salary of employees

```
SELECT AVG(sal) FROM emp;
```

NULL Values in SQL

◆ What is NULL?

In SQL, **NULL** means:

- Missing value
- Unknown value
- Not applicable value

NULL is **not 0**

NULL is **not a blank space** (' ')

NULL cannot be compared using = or !=

```
SELECT * FROM emp WHERE comm IS NULL;
```

To Check **NOT NULL**

```
SELECT * FROM emp WHERE comm IS NOT NULL;
```

NULL Values

Arithmetic with NULL

If a value contains NULL, the result becomes NULL.

```
SELECT sal + comm FROM emp;
```

If comm is NULL → result is NULL.

Using NVL

NVL() replaces NULL with a value.

```
SELECT ename, sal + NVL(comm,0) AS total_salary  
FROM emp;
```

If commission is NULL, it treats it as 0.

Using COALESCE (Standard SQL)

```
SELECT ename, COALESCE(comm,0)  
FROM emp;
```

NULL Values

Aggregate functions ignore NULL values:

- SUM(), AVG(), MAX(), MIN()

Important Points

- NULL means unknown/missing value.
- Use IS NULL or IS NOT NULL.
- NULL in arithmetic → result becomes NULL.
- Use NVL() or COALESCE() to handle NULL.
- Aggregate functions ignore NULL (except COUNT(*)).

GROUP BY Clause

The SQL GROUP BY clause is used to arrange identical data into groups based on one or more columns. It is commonly used with aggregate functions like COUNT(), SUM(), AVG(), MAX(), and MIN() to perform calculations on each group of data.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1, column2;
```

Display department-wise total salary.

```
SELECT deptno, SUM(sal) AS total_salary
FROM emp
GROUP BY deptno;
```

GROUP BY Clause

Display department-wise average salary.

```
SELECT deptno, AVG(sal) AS avg_salary  
FROM emp  
GROUP BY deptno;
```

Count number of employees in each department.

```
SELECT deptno, COUNT(*) AS total_employees  
FROM emp  
GROUP BY deptno;
```

Display job-wise maximum salary.

```
SELECT job, MAX(sal) AS max_salary  
FROM emp  
GROUP BY job;
```

Display department-wise minimum salary.

GROUP BY Clause

Display departments having more than 3 employees.

```
SELECT deptno, COUNT (*)  
FROM emp  
GROUP BY deptno  
HAVING COUNT (*) > 3;
```

Display departments whose total salary is greater than 10000.

```
SELECT deptno, SUM(sal)  
FROM emp  
GROUP BY deptno  
HAVING SUM(sal) > 10000;
```

Display job-wise average salary in department 20.

Display number of employees under each manager.

Display department-wise total salary in descending order.

GROUP BY Clause & HAVING

Display department-wise total salary and show only departments where total salary is greater than 10000.

```
SELECT deptno, SUM(sal) AS total_salary  
FROM emp  
GROUP BY deptno  
HAVING SUM(sal) > 10000;
```

Display departments having more than 3 employees.

```
SELECT deptno, COUNT(*) AS total_employees  
FROM emp  
GROUP BY deptno  
HAVING COUNT(*) > 3;
```

GROUP BY Clause & HAVING

Display department-wise maximum salary where maximum salary is greater than 3000.

```
SELECT deptno, MAX(sal) AS max_salary  
FROM emp  
GROUP BY deptno  
HAVING MAX(sal) > 3000;
```

Display departments where minimum salary is less than 1000.

```
SELECT deptno, MIN(sal) AS min_salary  
FROM emp  
GROUP BY deptno  
HAVING MIN(sal) < 1000;
```

GROUP BY Clause & HAVING

Display job-wise total salary only for department 20 and show results where total salary is greater than 5000.

```
SELECT job, SUM(sal) AS total_salary
FROM emp
WHERE deptno = 20
GROUP BY job
HAVING SUM(sal) > 5000;
```

Display managers having more than 2 employees under them.

```
SELECT mgr, COUNT(*) AS total_employees
FROM emp
WHERE mgr IS NOT NULL
GROUP BY mgr
HAVING COUNT(*) > 2;
```

GROUP BY Clause & HAVING

Display department-wise total salary for departments 10 and 20 only, where total salary is greater than 8000. Arrange results in descending order of total salary.

Display job-wise average salary for employees earning more than 1000, where average salary is greater than 2000. Sort by average salary.

Display department-wise number of employees and show only departments having more than 3 employees and total salary greater than 9000. Sort by employee count descending.

Display manager-wise total salary of employees under each manager (ignore NULL managers). Show only managers where total salary is greater than 5000. Sort by total salary descending.

Display department-wise maximum salary for employees hired after 01-JAN-1982. Show only departments where maximum salary is greater than 3000. Sort by department number.

Display job-wise total salary and average salary for department 30 only. Show only jobs where average salary is greater than 1500 and total salary is greater than 4000. Sort by average salary descending.

GROUP BY Clause & HAVING

```
SELECT deptno, SUM(sal) AS total_salary
FROM emp
WHERE deptno IN (10,20)
GROUP BY deptno
HAVING SUM(sal) > 8000
ORDER BY total_salary DESC;
```

```
SELECT job, AVG(sal) AS avg_salary
FROM emp
WHERE sal > 1000
GROUP BY job
HAVING AVG(sal) > 2000
ORDER BY avg_salary;
```

```
SELECT deptno, COUNT(*) AS total_emp, SUM(sal) AS
total_salary
FROM emp
GROUP BY deptno
HAVING COUNT(*) > 3 AND SUM(sal) > 9000
ORDER BY total_emp DESC;
```

GROUP BY Clause & HAVING

```
SELECT mgr, SUM(sal) AS total_salary
FROM emp
WHERE mgr IS NOT NULL
GROUP BY mgr
HAVING SUM(sal) > 5000
ORDER BY total_salary DESC;
```

```
SELECT deptno, MAX(sal) AS max_salary
FROM emp
WHERE hiredate > '01-JAN-1982'
GROUP BY deptno
HAVING MAX(sal) > 3000
ORDER BY deptno;
```

```
SELECT job, SUM(sal) AS total_salary, AVG(sal) AS avg_salary
FROM emp
WHERE deptno = 30
GROUP BY job
HAVING AVG(sal) > 1500 AND SUM(sal) > 4000
ORDER BY avg_salary DESC;
```

SQL Joins are used to combine data from two or more tables based on a related column. They help in:

- Retrieving connected data stored across multiple tables.
- Matching records using common columns.
- Improving data analysis by combining related information.
- Creating meaningful result sets from separate tables.

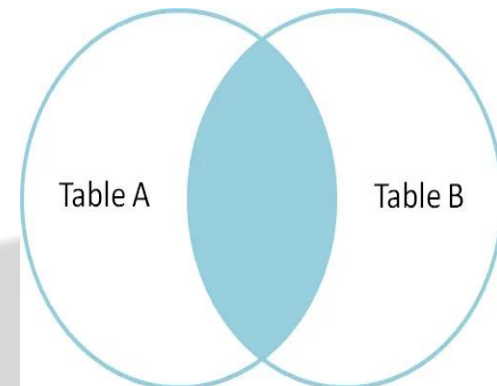
Types of SQL Joins (**Inner, Left, Right and Full Join**)

SQL joins are categorized into different types based on how rows from two tables are matched and combined.

1. SQL INNER JOIN

INNER JOIN is used to retrieve rows where matching values exist in both tables. It helps in:

- Combining records based on a related column.
- Returning only matching rows from both tables.
- Excluding non-matching data from the result set.
- Ensuring accurate data relationships between tables.



Joins

EMP(EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)
DEPT(DEPTNO, DNAME, LOC)

```
SELECT table1.column1,table1.column2,table2.column1,....  
FROM table1  INNER JOIN  table2  
ON  table1.matching_column = table2.matching_column;
```

Display employee name with department name.

```
SELECT e.ename, d.dname  
FROM emp e  
INNER JOIN dept d  
ON e.deptno = d.deptno;
```

EQUI JOIN/ INNER JOIN

EQUI JOIN in SQL

An **Equi Join** is a type of **INNER JOIN** where tables are joined using the **equality operator (=)**.

- It matches rows when values in the common column are equal.

EMP(EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)

DEPT(DEPTNO, DNAME, LOC)

Common column → DEPTNO

Syntax(Old Style – Oracle)

```
SELECT column_list
FROM table1, table2
WHERE table1.column = table2.column;
```

Syntax (Modern ANSI Style – Recommended)

```
SELECT column_list
FROM table1
JOIN table2
ON table1.column = table2.column;
```

Display employee name, salary, and department location.

```
SELECT e.ename, e.sal, d.loc
FROM emp e, dept d
WHERE e.deptno = d.deptno;
```

```
SELECT e.ename, e.sal, d.loc
FROM emp e JOIN dept d
ON e.deptno = d.deptno;
```

Non-Equi Joins

SALGRADE(GRADE, LOSAL, HISAL)

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

EMP(EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)

DEPT(DEPTNO, DNAME, LOC)

Display employee name, salary, and salary grade.

```
SELECT e.ename, e.sal, s.grade
```

```
FROM emp e
```

```
JOIN salgrade s
```

```
ON e.sal BETWEEN s.losal AND s.hisal;
```

✓ Here we use BETWEEN (not =).

✓ So this is a **Non-Equi Join**.

Joins

Old Model	New ANSI Model
Uses comma (,)	Uses JOIN keyword
Join condition in WHERE	Join condition in ON
Oracle traditional style	Standard SQL style

LEFT OUTER JOIN

LEFT OUTER JOIN

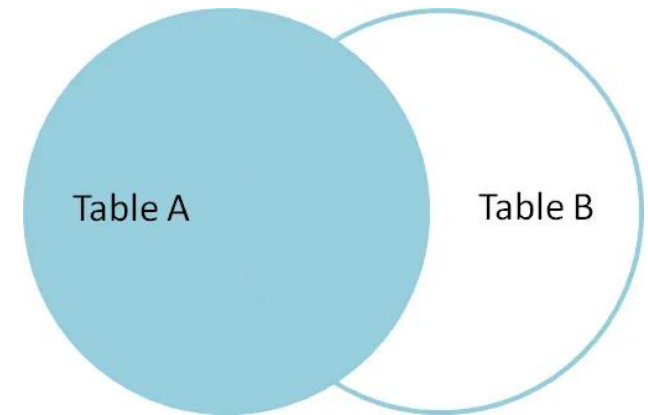
Returns all rows from left table + matching rows from right table.

If no match → NULL values.

Example:

Display all employees and their department names.

```
SELECT e.ename, d.dname  
FROM emp e  
LEFT JOIN dept d  
ON e.deptno = d.deptno;
```



SQL LEFT JOIN

LEFT JOIN is used to retrieve all rows from the left table and matching rows from the right table. It helps in:

- Returning all records from the left table.
- Showing matching data from the right table.
- Displaying NULL values where no match exists in the right table.
- Performing outer joins, also known as LEFT OUTER JOIN.

RIGHT OUTER JOIN

RIGHT OUTER JOIN

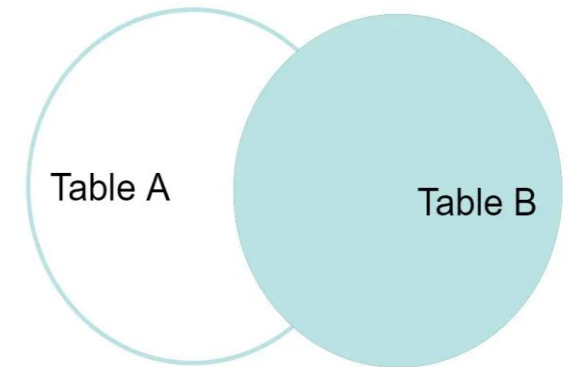
Returns all rows from right table + matching rows from left table.

Example:

Display all departments and employee names.

```
SELECT e.ename, d.dname  
FROM emp e  
RIGHT JOIN dept d  
ON e.deptno = d.deptno;
```

✓ Shows all departments even if no employees.



SQL RIGHT JOIN

RIGHT JOIN is used to retrieve all rows from the right table and the matching rows from the left table. It helps in:

- Returning all records from the right-side table.
- Showing matching data from the left-side table.
- Displaying NULL values where no match exists in the left table.
- Performing outer joins, also known as RIGHT OUTER JOIN.

FULL OUTER JOIN

FULL OUTER JOIN

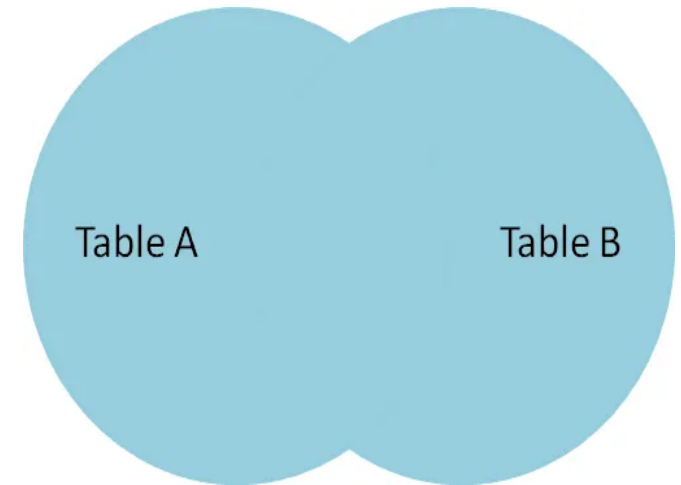
Returns all rows from both tables.

If no match → NULL values.

Example:

```
SELECT e.ename, d.dname  
FROM emp e  
FULL JOIN dept d  
ON e.deptno = d.deptno;
```

✓ Shows all employees and all departments.



SQL FULL JOIN

FULL JOIN is used to combine the results of both LEFT JOIN and RIGHT JOIN. It helps in:

- Returning all rows from both tables.
- Showing matching records from each table.
- Displaying NULL values where no match exists in either table.
- Providing complete data from both sides of the join.

Natural Join JOIN

SQL Natural Join

A Natural Join is a type of INNER JOIN that automatically joins two tables based on columns with the same name and data type. It returns only the rows where the values in the common columns match.

- It joins tables using common columns with the same name.
- It returns only rows where values in those columns match.
- The common column appears only once in the result.

Example: Look at the two tables below:

Employee Table:

Emp_id	Emp_name	Dept_id
1	Ram	10
2	Jon	30
3	Bob	50

Department Table:

Dept_id	Dept_name
10	IT
30	HR
40	TIS

Example: Find all Employees and their respective departments.

```
SELECT
    Emp_name,
    Dept_name
FROM Employee
NATURAL JOIN Department;
```

Output:

Emp_name	Dept_name
Ram	IT
Jon	HR

SELF JOIN

Joining a table with itself.

Example:

Display employee names and their manager name.

```
SELECT e.ename AS employee,  
       m.ename AS manager  
FROM emp e  
JOIN emp m  
ON e.mgr = m.empno;
```

✓ Used when table references itself.

CROSS JOIN

CROSS JOIN

Returns Cartesian product (all combinations).

Example:

```
SELECT e.ename, d.dname  
FROM emp e  
CROSS JOIN dept d;
```

✓ If EMP has 14 rows and DEPT has 4 rows → 56 rows.

CROSS JOIN

Join Type	Returns
INNER JOIN	Matching rows only
LEFT JOIN	All left + matching right
RIGHT JOIN	All right + matching left
FULL JOIN	All rows from both
CROSS JOIN	Cartesian product
SELF JOIN	Table joined with itself

A nested query (or subquery) is a SQL query written inside another query to solve complex data problems. The inner query executes first and passes its result to the main query.

- It allows one query to use the result of another query.
- The inner query runs first, then the outer query processes its result.
- Used for filtering, updating, and retrieving data from related tables.

A Subquery (Nested Query) is a query written inside another SQL query.

- The inner query executes first.
- Its result is used by the outer query.

Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name operator  
      (SELECT column_name FROM table_name);
```

Types of Subqueries

- Single Row Subquery
- Multiple Row Subquery
- Correlated Subquery
- Nested Subquery in SELECT
- Subquery in FROM clause

EMP(EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)
DEPT(DEPTNO, DNAME, LOC)

Single Row Subquery

Returns only **one value**. (The inner query does NOT use any column from the outer query.
so the **subquery can run independently**.)

Display employees earning more than the average salary.

```
SELECT ename, sal
```

```
FROM emp
```

```
WHERE sal > (SELECT AVG(sal) FROM emp);
```

Nested Queries

Comparison with Correlated Subquery

Type	Execution
Normal Subquery	Inner query runs once
Correlated Subquery	Inner query runs for each row

Multiple Row Subquery

Returns multiple values.

Uses IN, ANY, ALL.

Display employees working in departments located in NEW YORK.

```
SELECT ename
```

```
FROM emp
```

```
WHERE deptno IN
```

```
    (SELECT deptno FROM dept WHERE loc = 'NEW YORK') ;
```

Correlated Subquery (The inner query uses a column from the outer query. So, the subquery must run for every row of the outer query.)

Inner query depends on outer query.

Runs once for each row.

Display employees earning more than average salary of their department.

```
SELECT e1.ename, e1.sal, e1.deptno
FROM emp e1
WHERE sal >
      (SELECT AVG(sal)
       FROM emp e2
        WHERE e1.deptno = e2.deptno);
```

✓ Inner query depends on outer query row.

Subquery in SELECT Clause

Display employee name and department name using subquery.

```
SELECT  ename,  
        (SELECT  dname  
         FROM    dept  
         WHERE   dept.deptno = emp.deptno) AS  
department  
FROM    emp;
```


Subquery in FROM Clause (Inline View)

Display departments whose total salary is greater than 10000.

```
SELECT deptno, total_salary
FROM (
    SELECT deptno, SUM(sal) AS total_salary
    FROM emp
    GROUP BY deptno
)
WHERE total_salary > 10000;
```

Nested Queries

Operator	Meaning
IN	Matches any value in list
ANY	Compare with any value
ALL	Compare with all values
EXISTS	Checks if subquery returns rows

Execution Order

- 1.Inner query executes first
- 2.Outer query executes next

Nested Queries

Display employees earning more than average salary.

```
SELECT ename, sal
FROM emp
WHERE sal > (SELECT AVG(sal) FROM emp);
```

Display employees working in department 10.

```
SELECT ename
FROM emp
WHERE deptno = (SELECT deptno FROM dept WHERE deptno = 10);
```

Display employees working in departments located in NEW YORK.

```
SELECT ename
FROM emp
WHERE deptno IN
      (SELECT deptno FROM dept WHERE loc = 'NEW YORK');
```

Display employees whose salary is equal to maximum salary.

```
SELECT ename, sal
FROM emp
WHERE sal = (SELECT MAX(sal) FROM emp);
```

Nested Queries

Display employees earning more than ANY employee in department 10.

```
SELECT ename, sal
FROM emp
WHERE sal > ANY
      (SELECT sal FROM emp WHERE deptno = 10);
```

Display employees earning more than ALL employees in department 20.

```
SELECT ename, sal
FROM emp
WHERE sal > ALL
      (SELECT sal FROM emp WHERE deptno = 20);
```

Display employees who earn the highest salary in their department.

```
SELECT e1.ename, e1.sal, e1.deptno
FROM emp e1
WHERE sal =
      (SELECT MAX(sal)
       FROM emp e2
       WHERE e1.deptno = e2.deptno);
```

Nested Queries

Display departments that have at least one employee earning more than 3000.

```
SELECT deptno, dname
FROM dept d
WHERE EXISTS
    (SELECT 1
     FROM emp e
     WHERE e.deptno = d.deptno
          AND e.sal > 3000);
```

Display employees who do not work in departments located in DALLAS.

```
SELECT ename
FROM emp
WHERE deptno NOT IN
    (SELECT deptno
     FROM dept
     WHERE loc = 'DALLAS');
```

Relational algebra

Relational algebra in DBMS is a procedural query language that acts as the theoretical foundation for SQL, providing a step-by-step method to manipulate and retrieve data from relational databases. a procedural query language that acts as the theoretical foundation for SQL, providing a step-by-step method to manipulate and retrieve data from relational databases. It uses operators (unary/binary) to take one or more relations as input and return a new relation, including fundamental operations like selection, projection, and join

Definition

Relational Algebra is a **set of operations that take one or two relations (tables) as input and produce another relation as output.**

Relational Algebra

Table 6.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
SELECT	Selects all tuples that satisfy the selection condition from a relation R .	$\sigma_{\langle \text{selection condition} \rangle}(R)$
PROJECT	Produces a new relation with only some of the attributes of R , and removes duplicate tuples.	$\pi_{\langle \text{attribute list} \rangle}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \bowtie_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 \star_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \star_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$ OR $R_1 \star R_2$
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$

Relational Algebra

INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2 ; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2 .	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Figure 6.9 shows a query tree for Query 2 (see Section 4.3.1): *For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.* This query is specified on the relational schema of Figure 3.5 and corresponds to the following relational algebra expression:

$$\pi_{Pnumber, Dnum, Lname, Address, Bdate}(((\sigma_{Plocation='Stafford'}(PROJECT)) \bowtie_{Dnum=Dnumber} (DEPARTMENT)) \bowtie_{Mgr_ssn=Ssn} (EMPLOYEE))$$

Display all employees

$\pi * (EMP)$

Display employee names

$\pi_{ENAME} (EMP)$

Employees with salary greater than 2000

$\sigma_{SAL > 2000} (EMP)$

Employees in department 10

$\sigma_{DEPTNO = 10} (EMP)$

Display employee name and salary

$\pi_{ENAME, SAL} (EMP)$

Relational Algebra

Employees working in department 20

$\pi_{ENAME} (\sigma_{DEPTNO = 20} (EMP))$

Employees whose salary is greater than 3000

$\pi_{ENAME, SAL} (\sigma_{SAL > 3000} (EMP))$

Display employee name and department name

$\pi_{ENAME, DNAME} (EMP \bowtie DEPT)$

Employees working in SALES department

$\pi_{ENAME} (\sigma_{DNAME = 'SALES'} (EMP \bowtie DEPT))$

Employees with salary between 2000 and 4000

$\sigma_{SAL \geq 2000 \wedge SAL \leq 4000} (EMP)$

Relational Algebra

Display employee name and department location

$\pi_{ENAME, LOC} (EMP \bowtie DEPT)$

Employees and their salary grade

$\pi_{ENAME, GRADE} (EMP \bowtie SALGRADE)$

Employees working in NEW YORK

$\pi_{ENAME} (\sigma_{LOC = 'NEW YORK'} (EMP \bowtie DEPT))$

Relational Algebra

Employees whose salary is greater than 3000 and department is RESEARCH

$\pi_{ENAME} (\sigma_{SAL > 3000 \wedge DNAME = 'RESEARCH'} (EMP \bowtie DEPT))$

Display employee name, department name, and salary grade

$\pi_{ENAME, DNAME, GRADE} (EMP \bowtie DEPT \bowtie SALGRADE)$

Employees whose salary grade is 4

$\pi_{ENAME} (\sigma_{GRADE = 4} (EMP \bowtie SALGRADE))$

Employees working in departments located in DALLAS

$\pi_{ENAME} (\sigma_{LOC = 'DALLAS'} (EMP \bowtie DEPT))$

Set Operator examples in Relational Algebra using EMP, DEPT, and SALGRADE tables

1. Employees working in department 10 or 20

π ENAME (σ DEPTNO = 10 (EMP))

$\cup$

π ENAME (σ DEPTNO = 20 (EMP))

2. Employees working in NEW YORK or DALLAS

π ENAME (σ LOC = 'NEW YORK' (EMP $\bowtie$ DEPT))

$\cup$

π ENAME (σ LOC = 'DALLAS' (EMP $\bowtie$ DEPT))

3. Employees with salary > 3000 or salary grade 4

π ENAME (σ SAL > 3000 (EMP))

$\cup$

π ENAME (σ GRADE = 4 (EMP $\bowtie$ SALGRADE))

Relational Algebra

4. Employees in department 10 and salary > 2000

π ENAME (σ DEPTNO = 10 (EMP))

$\cap$

π ENAME (σ SAL > 2000 (EMP))

5. Employees working in SALES department and salary > 1500

π ENAME (σ DNAME = 'SALES' (EMP $\bowtie$ DEPT))

$\cap$

π ENAME (σ SAL > 1500 (EMP))

6. Employees with salary grade 3 and working in department 20

π ENAME (σ GRADE = 3 (EMP $\bowtie$ SALGRADE))

$\cap$

π ENAME (σ DEPTNO = 20 (EMP))

Relational Algebra

7. Employees in department 10 but not department 20

π ENAME (σ DEPTNO = 10 (EMP))

–

π ENAME (σ DEPTNO = 20 (EMP))

8. Employees working in SALES but not in NEW YORK

π ENAME (σ DNAME = 'SALES' (EMP $\bowtie$ DEPT))

–

π ENAME (σ LOC = 'NEW YORK' (EMP $\bowtie$ DEPT))

9. Employees with salary > 2000 but not salary grade 4

π ENAME (σ SAL > 2000 (EMP))

–

π ENAME (σ GRADE = 4 (EMP $\bowtie$ SALGRADE))

Relational Algebra

1. Total salary of all employees

$\Upsilon \text{ SUM(SAL) (EMP)}$ or $\mathcal{F} \text{ SUM(SAL) (EMP)}$

2. Average salary of employees

$\Upsilon \text{ AVG(SAL) (EMP)}$

3. Number of employees in each department

$\Upsilon \text{ DEPTNO, COUNT(EMPNO) (EMP)}$

4. Maximum salary in each department

$\Upsilon \text{ DEPTNO, MAX(SAL) (EMP)}$

Relational Algebra

Average salary of department 10 employees

$\mathcal{F} \text{ AVG}(\text{SAL}) (\sigma \text{ DEPTNO} = 10 (\text{EMP}))$

Total salary of employees working in SALES department

$\mathcal{F} \text{ SUM}(\text{SAL}) (\sigma \text{ DNAME} = \text{'SALES'} (\text{EMP} \bowtie \text{DEPT}))$

or

$\mathcal{V} \text{ SUM}(\text{SAL}) (\sigma \text{ DNAME} = \text{'SALES'} (\text{EMP} \bowtie \text{DEPT}))$

Relational Algebra

The DIVISION operation can be expressed as a sequence of π , $\times$, and $-$ operations as follows:

$$T_1 \leftarrow \pi_Y(R)$$

$$T_2 \leftarrow \pi_Y((S \times T_1) - R)$$

$$T \leftarrow T_1 - T_2$$

Retrieve the names of employees who work on *all* the projects that 'John Smith' works on

$$R \div S$$

Relational Algebra

R(A,B)		S(A)
A	B	A
a1	b1	a1
a2	b1	a2
a3	b1	a3
a4	b1	
a1	b2	
a3	b2	
a2	b3	
a3	b3	
a4	b3	
a1	b4	
a2	b4	
a3	b4	

Step 1

$$T_1 \leftarrow \pi_B(R)$$

Project **B** from relation **R**.



B
b1
b2
b3
b4

So

$$T_1 = \{ b1, b2, b3, b4 \}$$

Step 2

Compute Cartesian product
 $S \times T_1$



A	B
a1	b1
a1	b2
a1	b3
a1	b4
a2	b1
a2	b2
a2	b3
a2	b4
a3	b1
a3	b2
a3	b3
a3	b4

Relational Algebra

$S \times T_1$

A	B
a1	b1
a1	b2
a1	b3
a1	b4
a2	b1
a2	b2
a2	b3
a2	b4
a3	b1
a3	b2
a3	b3
a3	b4

$R(A,B)$

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

Step 3

$(S \times T_1) - R$

Remove tuples already present in **R**.

Missing tuples are:



A	B
a1	b3
a2	b2

Step 4

$T_2 \leftarrow \pi_B((S \times T_1) - R)$

Project **B**.



B
b3
b2

Step 5

Final result

$T = T_1 - T_2$

$T_1 = \{ b1, b2, b3, b4 \}$

$T_2 = \{ b2, b3 \}$

Result:



B
b1
b4

Final Answer

$R \div S$

B
b1
b4

Relational Calculus (TRC, DRC): Tuple Relational Calculus



Relational Calculus is a **non-procedural query language** used in database systems. It describes **what data we want**, not **how to get it** (unlike Relational Algebra which is procedural).

Tuple Relational Calculus (TRC)

- Uses **tuple variables**
- Represents queries using tuples.

General form/Syntax: The basic syntax of TRC is as follows:

$$\{ t \mid P(t) \}$$

- **t**: Tuple variable (row placeholder)
- **P(t)**: Predicate condition to satisfy
- **{}**: Denotes a set of result tuples

Logical Operators in TRC:

- $\wedge$: AND
- $\vee$: OR
- $\neg$: NOT

Quantifiers:

- $\exists t \in r (Q(t)) \rightarrow$ There exists a tuple t in relation r satisfying predicate $Q(t)$
- $\forall t \in r (Q(t)) \rightarrow$ For all tuples t in relation r , predicate $Q(t)$ holds

Relational Algebra TRC

Find employees working in department 10.

EMP(EmpNo, Name, Job, Sal, DeptNo)

Query:

$$\{t \mid t \in EMP \wedge t.DeptNo = 10\}$$

Meaning:

Return all tuples **t** from EMP where **DeptNo = 10**.

To retrieve the names of all employees who earn more than 50,000

$$\{ t \mid Emp(t) \wedge t.Salary > 50000 \}$$

Explanation:

- Employees(t) means t is a tuple from the Employees table.
- $\wedge$ (AND) is used to add a condition on salary.
- The result is a set of tuples where each employee earns more than 50,000.

TRC is non-procedural - it specifies what data to retrieve, not how to retrieve it.

While expressive, TRC is more abstract and mainly used in academic or theoretical contexts, not practical database systems.

Relational Algebra TRC

Symbols Used in Relational Calculus

Symbol	Meaning
$\in$	belongs to
$\wedge$	AND
$\vee$	OR
$\neg$	NOT
$\exists$	Exists
$\forall$	For all

Relational Algebra TRC

1. List all employees $\{e \mid e \in EMP\}$ Return all tuples **e** from EMP.
2. Employees working in department 10 $\{e \mid e \in EMP \wedge e.DeptNo = 10\}$
3. Employees with salary greater than 3000 $\{e \mid e \in EMP \wedge e.Sal > 3000\}$
4. Employees working in SALES department
 $\{e \mid e \in EMP \wedge \exists d(d \in DEPT \wedge d.DeptNo = e.DeptNo \wedge d.DName = "SALES")\}$
Meaning: There exists a department **d** named SALES whose DeptNo matches employee.
5. Employees working in NEW YORK location
 $\{e \mid e \in EMP \wedge \exists d(d \in DEPT \wedge d.DeptNo = e.DeptNo \wedge d.Loc = "NEWYORK")\}$
6. Employees with salary grade 1
 $\{e \mid e \in EMP \wedge \exists s(s \in SALGRADE \wedge e.Sal \geq s.Losal \wedge e.Sal \leq s.Hisal \wedge s.Grade = 1)\}$
7. Employees whose salary belongs to some grade
 $\{e \mid e \in EMP \wedge \exists s(s \in SALGRADE \wedge e.Sal \geq s.Losal \wedge e.Sal \leq s.Hisal)\}$

Relational Algebra TRC

8. Employees working in departments located in DALLAS

$$\{e \mid e \in EMP \wedge \exists d(d \in DEPT \wedge d.DeptNo = e.DeptNo \wedge d.Loc = "DALLAS")\}$$

9. Employees whose salary is greater than all employees in department 10

$$\{e \mid e \in EMP \wedge \forall x(x \in EMP \wedge x.DeptNo = 10 \rightarrow e.Sal > x.Sal)\}$$

Uses FOR ALL ($\forall$).

10. Employees who work in departments that exist in DEPT table

$$\{e \mid e \in EMP \wedge \exists d(d \in DEPT \wedge d.DeptNo = e.DeptNo)\}$$

11. Employees whose salary falls in grade 3

$$\{e \mid e \in EMP \wedge \exists s(s \in SALGRADE \wedge s.Grade = 3 \wedge e.Sal \geq s.Losal \wedge e.Sal \leq s.Hisal)\}$$

12. Employees who earn more than every employee in department 20

$$\{e \mid e \in EMP \wedge \forall x(x \in EMP \wedge x.DeptNo = 20 \rightarrow e.Sal > x.Sal)\}$$

Relational Algebra DRC

In **Domain Relational Calculus (DRC)** we use **attribute variables** instead of tuple variables.
General Form

$$\{\langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n)\}$$

Where

• x_1, x_2 are **domain variables** (attribute values)

- **EMP**(EmpNo, EName, Job, Mgr, HireDate, Sal, Comm, DeptNo)
- **DEPT**(DeptNo, DName, Loc)
- **SALGRADE**(Grade, Losal, Hisal)

EMP(EmpNo, EName, Job, Mgr, HireDate, Sal, Comm, DeptNo)

So in the predicate

$$EMP(e, n, j, m, h, s, c, dd)$$

the variables represent:

Relational Algebra DRC

1. Employee names working in department 10

Method 1 (constant inside relation)

$$\{\langle n \rangle \mid \exists e, j, m, h, s, c \text{ EMP}(e, n, j, m, h, s, c, 10)\}$$

Method 2 (variable + condition) ✓ more formal

$$\{\langle n \rangle \mid \exists e, j, m, h, s, c, dd (\text{EMP}(e, n, j, m, h, s, c, dd) \wedge dd = 10)\}$$

2. Employee names with salary greater than 3000

$$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d (\text{EMP}(e, n, j, m, h, s, c, d) \wedge s > 3000)\}$$

3. Employees working in SALES department

$$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d, dd, dn, l (\text{EMP}(e, n, j, m, h, s, c, d) \wedge \text{DEPT}(dd, dn, l) \wedge dn = \text{'SALES'} \wedge d = dd)\}$$

4. Employees working in NEW YORK location

$$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d, dd, dn, l (\text{EMP}(e, n, j, m, h, s, c, d) \wedge \text{DEPT}(dd, dn, l) \wedge l = \text{'NEW YORK'} \wedge d = dd)\}$$

5. Employees whose salary falls in Grade 1

$$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d, g, lo, hi (\text{EMP}(e, n, j, m, h, s, c, d) \wedge \text{SALGRADE}(g, lo, hi) \wedge g = 1 \wedge s \geq lo \wedge s \leq hi)\}$$

Relational Algebra DRC

6. Employees whose salary belongs to any grade

$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d, g, lo, hi (EMP(e, n, j, m, h, s, c, d) \wedge SALGRADE(g, lo, hi) \wedge s \geq lo \wedge s \leq hi)\}$

Meaning:

Employee salary must fall between **Losal** and **Hisal** of some grade.

7. Employees working in DALLAS

$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d, dd, dn, l (EMP(e, n, j, m, h, s, c, d) \wedge DEPT(dd, dn, l) \wedge l = 'DALLAS' \wedge d = dd)\}$

8. Employees with salary greater than all employees in department 10

$\{\langle n \rangle \mid \exists e, j, m, h, s, c, d (EMP(e, n, j, m, h, s, c, d) \wedge \forall e_2, n_2, j_2, m_2, h_2, s_2, c_2 ((EMP(e_2, n_2, j_2, m_2, h_2, s_2, c_2, 10)) \rightarrow s > s_2))\}$

PL/SQL is a **procedural extension of SQL** developed by the Oracle Corporation for the Oracle Database.

It allows us to **combine SQL statements with programming features** like variables, loops, conditions, and procedures.

So, PL/SQL helps in writing **complete database programs inside the database**.

Why PL/SQL is Used

SQL alone can only **query and modify data**, but it cannot perform programming logic.

PL/SQL adds:

- Variables
- Conditions (IF)
- Loops
- Procedures
- Functions
- Triggers
- Exception handling

This makes database operations **faster and more secure**.

Structure of a PL/SQL Block

A PL/SQL program is written as a **block**.

Basic structure:

```
DECLARE
    -- variable declaration
BEGIN
    -- executable statements
EXCEPTION
    -- error handling
END;
```

Section	Purpose
DECLARE	Declare variables
BEGIN	Write SQL and logic
EXCEPTION	Handle errors
END	End of block

Simple PL/SQL Example

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');
END;
/
```

Output:

Welcome to PL/SQL

PL/SQL Block with Variables

```
DECLARE
    name VARCHAR2(20) := 'Ravi';
BEGIN
    DBMS_OUTPUT.PUT_LINE('Customer Name: ' || name);
END;
/
```

PL/SQL with SQL Query

Example: Display employee salary.

```
DECLARE
```

```
    salary NUMBER;
```

```
BEGIN
```

```
    SELECT sal INTO salary
```

```
    FROM emp
```

```
    WHERE empno = 7369;
```

```
    DBMS_OUTPUT.PUT_LINE('Salary = ' || sal);
```

```
END;
```

```
/
```


Control Structures in PL/SQL

IF Statement

```
IF balance > 50000 THEN  
    DBMS_OUTPUT.PUT_LINE('High Balance');  
END IF;
```

LOOP

```
FOR i IN 1..5 LOOP  
    DBMS_OUTPUT.PUT_LINE(i);  
END LOOP;
```

Total Control Structures in PL/SQL: 7

1. IF
2. IF-ELSE
3. IF-ELSIF-ELSE
4. LOOP
5. WHILE LOOP
6. FOR LOOP
7. GOTO

IF Statement

Executes a block if the condition is **true**.

```
IF condition THEN  
    statements;  
END IF;
```

Example

```
DECLARE  
    balance NUMBER := 60000;  
BEGIN  
    IF balance > 50000 THEN  
        DBMS_OUTPUT.PUT_LINE('High Balance');  
    END IF;  
END;  
/
```

IF–ELSE Statement

Executes **one block** if condition is true, otherwise another block.

```
IF condition THEN
    statements1;
ELSE
    statements2;
END IF;
```

Example

```
DECLARE
    balance NUMBER := 20000;
BEGIN
    IF balance > 50000 THEN
        DBMS_OUTPUT.PUT_LINE('High Balance');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Low Balance');
    END IF;
END;
/
```

IF–ELSIF–ELSE Statement

Used when there are **multiple conditions**.

```
IF condition1 THEN
    statements1;
ELSIF condition2 THEN
    statements2;
ELSE
    statements3;
END IF;
```

Example

```
DECLARE
    marks NUMBER := 75;
BEGIN
    IF marks >= 80 THEN
        DBMS_OUTPUT.PUT_LINE('Distinction');
    ELSIF marks >= 60 THEN
        DBMS_OUTPUT.PUT_LINE('First Class');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Pass');
    END IF;
END;
/
```

Basic LOOP

LOOP

statements;

EXIT WHEN condition;

END LOOP;

Example

DECLARE

i NUMBER := 1;

BEGIN

LOOP

DBMS_OUTPUT.PUT_LINE(i);

i := i + 1;

EXIT WHEN i > 5;

END LOOP;

END;

/

WHILE LOOP

Executes **while the condition is true**.

```
WHILE condition LOOP  
    statements;  
END LOOP;
```

Example

```
DECLARE  
    i NUMBER := 1;  
BEGIN  
    WHILE i <= 5 LOOP  
        DBMS_OUTPUT.PUT_LINE(i);  
        i := i + 1;  
    END LOOP;  
END;  
/
```

FOR LOOP

Used when the **number of iterations is known**.

```
FOR counter IN start..end LOOP  
    statements;  
END LOOP;
```

Example

```
BEGIN  
    FOR i IN 1..5 LOOP  
        DBMS_OUTPUT.PUT_LINE(i);  
    END LOOP;  
END;  
/
```

Sequential Control (GOTO)

Used to **transfer control** to another part of the program.

GOTO Statement

GOTO label;

Example

```
DECLARE
    i NUMBER := 1;
BEGIN
    <<start>>
    DBMS_OUTPUT.PUT_LINE(i);
    i := i + 1;

    IF i <= 5 THEN
        GOTO start;
    END IF;
END;
/
```


Syntax of Procedure

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter datatype)
IS
    variable_declaration datatype;
BEGIN
    -- SQL statements
EXCEPTION
    -- exception handling
END;
/
```

Procedure is used to **perform an action** but **does not return a value**.

Display Employee Salary

```
CREATE OR REPLACE PROCEDURE emp_sal
(p_empno NUMBER)
IS
    v_sal NUMBER;
BEGIN
    SELECT sal INTO v_sal
    FROM emp
    WHERE empno = p_empno;

    DBMS_OUTPUT.PUT_LINE('Salary = ' || v_sal);
END;
/
```

Execute Procedure

```
SET SERVEROUTPUT ON
EXEC emp_sal(7369);
```

Output

```
Salary = 800
```

Update Employee Salary)

```
CREATE OR REPLACE PROCEDURE update_sal
(p_empno NUMBER, p_newsala NUMBER)
IS
BEGIN
    UPDATE emp
    SET sal = p_newsala
    WHERE empno = p_empno;

    COMMIT;
END;
/
```

Syntax of Function

```
CREATE OR REPLACE FUNCTION function_name  
  (parameter datatype)  
  RETURN datatype  
IS  
    variable datatype;  
BEGIN  
    -- SQL statements  
    RETURN value;  
END;  
/
```

Function **returns a value.**

Return Employee Salary

```
CREATE OR REPLACE FUNCTION get_sal  
(p_empno NUMBER)  
RETURN NUMBER  
IS  
v_sal NUMBER;  
BEGIN  
SELECT sal INTO v_sal  
FROM emp  
WHERE empno = p_empno;  
  
RETURN v_sal;  
END;  
/
```

Execute Function

```
SELECT get_sal(7369) FROM dual;
```

Output

800

Example Function (Annual Salary)

```
CREATE OR REPLACE FUNCTION annual_sal
(p_empno NUMBER)
RETURN NUMBER
IS
    v_sal NUMBER;
BEGIN
    SELECT sal*12 INTO v_sal
    FROM emp
    WHERE empno = p_empno;

    RETURN v_sal;
END;
/
```

Execute

```
SELECT annual_sal(7369) FROM dual;
```

Difference Between Procedure and Function

Procedure	Function
Does not return value	Returns value
Called using EXEC	Used in SELECT
Used for actions	Used for calculations

Assertions in PL/SQL

In databases, an **Assertion** is a **constraint that ensures a condition is always true for the entire database**.

It is used to **maintain data integrity** by checking a condition before allowing changes in the database.

Assertions are part of the **SQL standard**, supported conceptually in systems like Oracle Database developed by Oracle Corporation.

Definition

An **Assertion** is a **global constraint on the database** that must always evaluate to **TRUE**. If the condition becomes **FALSE**, the database **rejects the operation**.

Syntax of Assertion (SQL Standard)

```
CREATE ASSERTION assertion_name  
CHECK (condition);
```

- **assertion_name** → name of the assertion
- **condition** → logical condition that must always be true

Example

Suppose a bank database has a rule:

Loan amount should never exceed 10,00,000

```
CREATE ASSERTION loan_limit
CHECK (NOT EXISTS (
    SELECT *
    FROM LOAN
    WHERE amount > 1000000
));
```

Meaning:

The database will **not allow any loan record where amount > 10,00,000.**

Example (Account Balance Rule)

Rule: Account balance must always be **greater than zero**

```
CREATE ASSERTION positive_balance
CHECK (NOT EXISTS (
    SELECT *
    FROM ACCOUNT
    WHERE balance < 0
));
```

Important Note in Oracle

Oracle does not directly support CREATE ASSERTION.

Instead, similar functionality is implemented using:

- CHECK constraints**
- TRIGGERS**
- Procedures**

Example using **CHECK constraint**

```
ALTER TABLE ACCOUNT  
ADD CONSTRAINT balance_check  
CHECK (balance >= 0);
```

Triggers in PL/SQL

A **Trigger** is a stored PL/SQL block that **automatically executes when a specific event occurs on a table or view**.

Triggers are widely used in databases like Oracle Database developed by Oracle Corporation.

They are mainly used to **enforce rules, maintain integrity, and audit changes** in the database.

Syntax of Trigger

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE | AFTER
INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- trigger statements
END;
/
```

Part	Meaning
BEFORE / AFTER	When the trigger executes
INSERT / UPDATE / DELETE	Event that activates trigger
ON table_name	Table where trigger is applied
FOR EACH ROW	Trigger runs for every row affected

Example Trigger (Display Message on Insert)

```
CREATE OR REPLACE TRIGGER emp_insert_trigger
BEFORE INSERT
ON emp
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('New employee record inserted');
END;
/
```

When a new employee is inserted, the message will appear.

Trigger to Prevent Salary Less Than 1000

```
CREATE OR REPLACE TRIGGER check_salary
BEFORE INSERT OR UPDATE
ON emp
FOR EACH ROW
BEGIN
    IF :NEW.sal < 1000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary too low');
    END IF;
END;
/
```

Explanation

:NEW.sal → new salary value being inserted or updated.

Trigger for Delete Operation

```
CREATE OR REPLACE TRIGGER emp_delete_trigger
AFTER DELETE
ON emp
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Employee record deleted');
END;
/
```

Trigger Using OLD and NEW Values

```
CREATE OR REPLACE TRIGGER salary_update
BEFORE UPDATE
ON emp
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Old Salary: ' || :OLD.sal);
    DBMS_OUTPUT.PUT_LINE('New Salary: ' || :NEW.sal);
END;
/
```

Types of Triggers

Type	Description
BEFORE Trigger	Executes before the event
AFTER Trigger	Executes after the event
Row-level Trigger	Executes for each row
Statement-level Trigger	Executes once for the statement

Events That Fire Triggers

Triggers can execute when:

- INSERT
- UPDATE
- DELETE

Advantages of Triggers

- Automatic execution
- Enforces business rules
- Maintains data integrity
- Useful for auditing changes